

# Machine Learning

## Regression Matrices

```
import numpy as np
#Dataset
x=np.array([1,2,3,4])
y=np.array([3,5,7,9])
m=0
b=0
learning_rate=0.01 # controls how much the model updates each step
epochs=1000 # Number of iterations
n=len(x)
for i in range(epochs):
    #predictions
    y_pred=m*x+b # current line equations
    #gradients
    dm=(-2/n)*np.sum(x*(y-y_pred))
    db=(-2/n)*np.sum(y-y_pred)
    # update parameters
    m=m-learning_rate*dm
    b=b-learning_rate*db #Move towards lower error
print("slope(m):",m)
print("Intercept(b):",b)

slope(m): 2.0048610156782556
Intercept(b): 0.9857080211211781

# OLS(Ordinary Least Squares)
import pandas as pd
from sklearn.linear_model import LinearRegression
#Sample Data
df=pd.DataFrame({
    'Experience': [1,2,3,4,5],
    'Salary': [30000,35000,40000,45000,50000]
})

#Input Feature(x)
x=df[['Experience']]
# Target Feature(y)
y=df ['Salary']
#Create Model
model=LinearRegression()
# Train Model(OLS Happens here)
model.fit(x,y)
#predict salary
predictions = model.predict(x)
#result
```

```

print("Slope:",model.coef_[0])
print("Intercept:",model.intercept_)
print("Predicted Salaries:",predictions)
#When fit() runs, OLS automatically calculates the best slope

Slope: 5000.0
Intercept: 25000.0
Predicted Salaries: [30000. 35000. 40000. 45000. 50000.]

new_experience = pd.DataFrame({ 'Experience': [6,7,8,9]})
#predict salary for new values
new_predictions = model.predict(new_experience)
print("Predicted Salaries for 6,7,8,9:")
print(new_predictions)

Predicted Salaries for 6,7,8,9:
[55000. 60000. 65000. 70000.]

import matplotlib.pyplot as plt
plt.figure(figsize=(8,5))
# Original data points
plt.scatter(df['Experience'], df['Salary'], color='blue',
label='Actual Data')
# Regression line
plt.plot(df['Experience'], predictions, color='red', label='Regression
Line')
# New predictions
plt.scatter(new_experience['Experience'], new_predictions,
            label='Predicted Points')
plt.xlabel("Experience (Years)")
plt.ylabel("Salary")
plt.title("Linear Regression: Experience vs Salary")
plt.legend()
plt.grid()
plt.show()

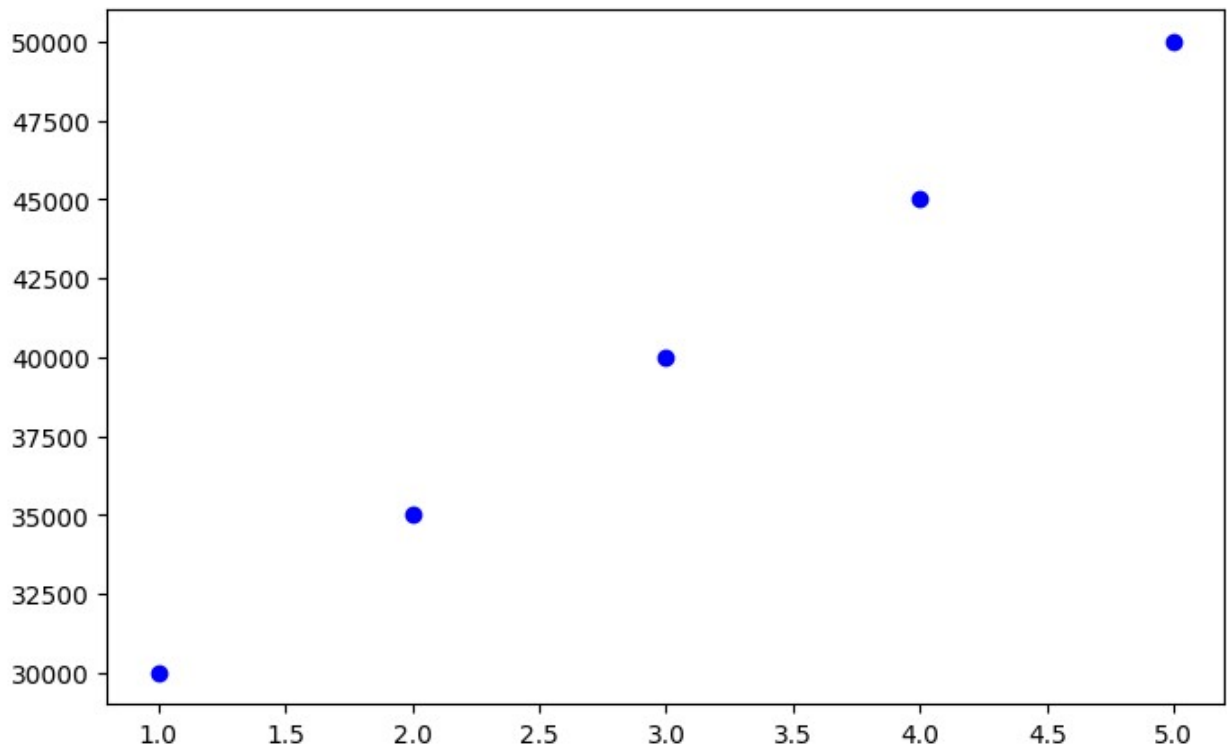
```

```

-----
-----
NameError                                Traceback (most recent call
last)
Cell In[5], line 6
      4 plt.scatter(df['Experience'], df['Salary'], color='blue',
label='Actual Data')
      5 # Regression line
----> 6 plt.plot(df['Experience'], predictions, color='red',
label='Regression Line')
      7 # New predictions
      8 plt.scatter(new_experience['Experience'], new_predictions,
      9                label='Predicted Points')

```

NameError: name 'predictions' is not defined



```
##Multiple Regression Code
import pandas as pd
from sklearn.linear_model import LinearRegression
# Dataset
df = pd.DataFrame({
    'Area': [1000, 1200, 1500, 1800, 2000],
    'Bedrooms': [2, 3, 3, 4, 4],
    'Age': [10, 8, 5, 4, 2],
    'Price': [300000, 350000, 450000, 500000, 600000]
})
# Features
X = df[['Area', 'Bedrooms', 'Age']] #Using multiple features.
# Target
y = df['Price']
# Model
model = LinearRegression()
# Train
model.fit(X, y) #Learns the relationship between all features and price.
# Prediction
pred = model.predict(X) #predict house price
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
```

```
##Multiple Regression Code (Practice)
```

```
##Multiple Regression Code (Practice)
```

```
import pandas as pd
from sklearn.linear_model import LinearRegression
new = pd.DataFrame({
    'Area': [2500, 3000, 5000, 6800, 2900],
    'Bedrooms': [4, 2, 2, 5, 5],
    'Age': [11, 7, 6, 5, 1],
    'Price': [390000, 550000, 660000, 590000, 880000]
})
X = df[['Area', 'Bedrooms', 'Age']] #Using multiple features.
y = df['Price']
model = LinearRegression()
model.fit(X, y) #Learns the relationship between all features and
price.
pred = model.predict(X) #predict house price
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
```

```
#VIF Code
```

```
import pandas as pd
from statsmodels.stats.outliers_influence import
variance_inflation_factor
# Features
X = df[['Area', 'Bedrooms', 'Age']]
# VIF Calculation
vif = pd.DataFrame()
vif['Feature'] = X.columns
vif['VIF'] = [
    variance_inflation_factor(X.values, i)
    for i in range(X.shape[1])
]
print(vif)
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
# Sample dataset
df = pd.DataFrame({
    'Hours': [1, 2, 3, 4, 5],
    'Marks': [10, 25, 45, 70, 100]
})
X = df[['Hours']]
y = df['Marks']
# Polynomial transformation
poly = PolynomialFeatures(degree=4)
```

```

X_poly = poly.fit_transform(X)
# Model training
model = LinearRegression()
model.fit(X_poly, y)
# Smooth curve generation (IMPORTANT STEP)
X_range = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
X_range = pd.DataFrame(X_range, columns=['Hours'])
X_range_poly = poly.transform(X_range)
# Predictions for curve
y_curve = model.predict(X_range_poly)
# Plotting
plt.scatter(X, y, color='red', label='Actual Data')
plt.plot(X_range, y_curve, color='blue', label='Polynomial Curve')
plt.title('Polynomial Regression Curve')
plt.xlabel('Hours')
plt.ylabel('Marks')
plt.legend()
plt.show()

```

Cell In[6], line 17

```

model = LinearRegression(
    ^

```

SyntaxError: '(' was never closed

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
X = np.array([1,2,3,4,5,6,7,8,9,10]).reshape(-1,1)
y = np.array([2,5,8,15,28,35,50,60,85,95])

# Degree 1
poly1 = PolynomialFeatures(degree=1)
X1 = poly1.fit_transform(X)

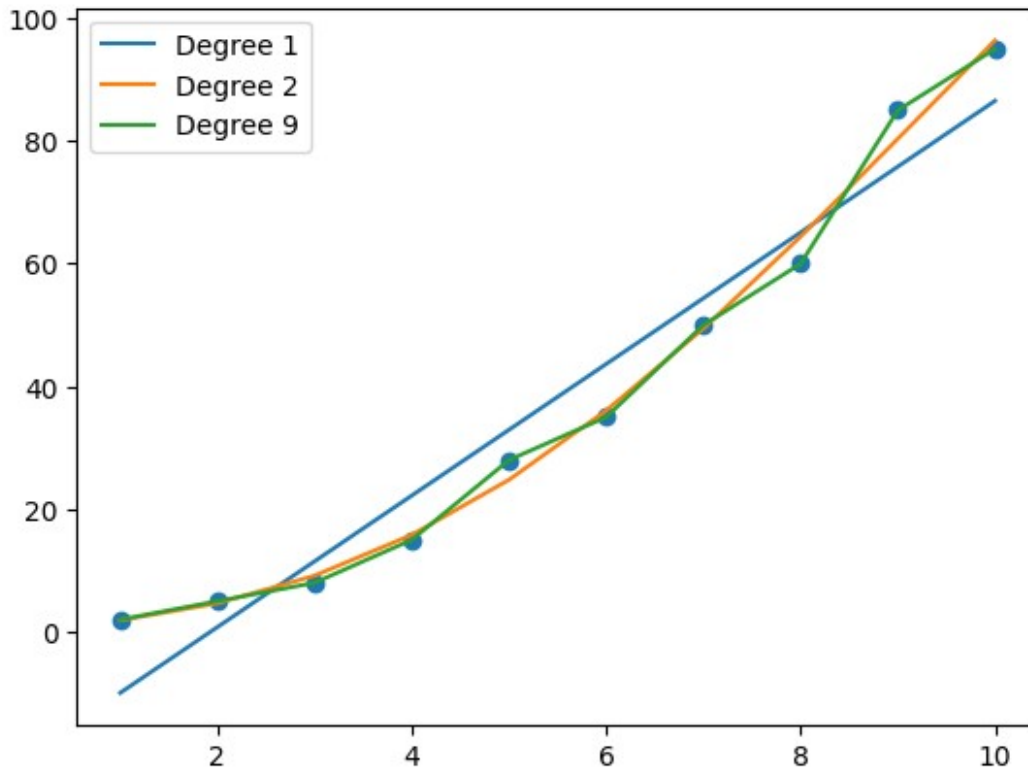
# Degree 4
poly4 = PolynomialFeatures(degree=4)
X4 = poly4.fit_transform(X)

# Degree 9
poly9 = PolynomialFeatures(degree=9)
X9 = poly9.fit_transform(X)
model1 = LinearRegression().fit(X1,y)
model2 = LinearRegression().fit(X4,y)
model9 = LinearRegression().fit(X9,y)

plt.scatter(X,y)
plt.plot(X, model1.predict(X1), label="Degree 1")
plt.plot(X, model2.predict(X4), label="Degree 2")

```

```
plt.plot(X, model9.predict(X9), label="Degree 9")
plt.legend()
plt.show()
```



#### ##Ridge Code

```
from sklearn.linear_model import Ridge
from sklearn.datasets import make_regression
X, y = make_regression(
    n_samples=100,
    n_features=5,
    noise=20,
    random_state=42
)
ridge = Ridge(alpha=1.0)
ridge.fit(X, y)
print("Coefficients:")
print(ridge.coef_)
```

Coefficients:

```
[62.4844981  98.07649112  57.01070157  52.31272567  35.35343002]
```

#### ##Lasso Code

```
from sklearn.linear_model import Lasso
from sklearn.datasets import make_regression
X, y = make_regression(
```

```

    n_samples=100,
    n_features=10,
    noise=20,
    random_state=42
)
lasso = Lasso(alpha=2.0)
lasso.fit(X, y)
print("Coefficients:")
print(lasso.coef_)

Coefficients:
[17.88753444 54.76076704  0.          60.9632585  90.0800089
 68.30461063
 81.35787563  4.03519677  2.86610797 67.99906953]

```

#### *##ElasticNet Code*

```

from sklearn.linear_model import ElasticNet
from sklearn.datasets import make_regression
X, y = make_regression(
    n_samples=100,
    n_features=10,
    noise=20,
    random_state=42
)
elastic = ElasticNet(
    alpha=3.0,
    l1_ratio=0.5
)
elastic.fit(X, y)
print("Coefficients:")
print(elastic.coef_)

Coefficients:
[10.65990986 22.10890939  0.49979146 25.9235125  33.08321402
 34.26097045
 35.7929364  -7.32756763  4.65692898 25.41452644]

```